

Contents

Preface	i
What is the goal of this book?	i
Which topics can you expect?	i
Who should read this book?	ii
What you must learn about Software Development yourself?	ii
What about the code examples and typographic conventions?	iii
Which literature is this book based on?	iii
Giving feedback?	iv
 1. Introduction to Software Craftmanship and Clean Code	 1
1.1 A Passion for Software Development	1
1.2 Manifesto for Software Craftmanship	3
1.3 Clean Code Developer	3
1.4 Boy Scout Rule	6
1.5 Broken Windows Theory	7
1.6 Cargo Cult Programming	8
1.7 Knowledge - Expertise	8
 2. Basics of Software Design	 9
2.1 Software Design Pyramid	9
2.2 Basic concepts of OOD	10
2.3 Goals of Software Design	13
2.4 Symptoms of <i>bad</i> design	14
2.5 Criteria for <i>good</i> design	14
2.6 Information Hiding	15
2.7 Cohesion	17
2.8 Coupling	18
2.9 Cohesion - Coupling	19
2.10 Big Ball of Mud	20
2.11 Architecture Principles	21
2.12 Cognitive Psychology and Architectural Principles	22
2.13 Layered Architecture	23
2.13.1 Use of Layered Architecture	23
2.13.2 Violated Layered Architecture	25
2.13.3 Horizontal Layering	26
2.13.4 Feature Based Layering - Single package	27
2.13.5 Feature Based Layering - Slices before layers	29

CONTENTS

2.13.6	Feature Based Layering – Hexagonal Architecture	30
2.13.7	The Java Module System	31
2.14	Architecture Documentation	32
2.15	Testing the Architecture and Design	34
2.16	Software Engineering Values	35
2.17	Team Charter	36
3.	Java Best Practices	37
3.1	Communicate through code	37
3.1.1	Use Java code conventions and avoid misinformation	37
3.1.2	Choose an expressive name and avoid mental mapping	38
3.1.3	Make differences clear with meaningful variable names	38
3.1.4	Use pronounceable names	38
3.1.5	Do not hurt the readers	39
3.1.6	Don't add redundant context	40
3.1.7	Don't add words without additional meaning	41
3.1.8	Don't use <i>and</i> or <i>or</i> in method names	42
3.1.9	Respect the order within classes	43
3.1.10	Group by line break	45
3.1.11	Prefer self-explanatory code instead of comments	46
3.1.12	Refactor step by step	48
3.2	Bad comments	49
3.2.1	Redundant comments	49
3.2.2	Misleading comments	49
3.2.3	Mandatory comments	50
3.2.4	Diary comments	50
3.2.5	Gossip	51
3.2.6	Position identifier	51
3.2.7	Write-ups and incidental remarks	52
3.2.8	Don't leave commented out code in your codebase	52
3.2.9	Rules for commenting	52
3.3	Classes and objects	53
3.3.1	Classes	53
3.3.2	Functions	53
3.3.3	Variables	54
3.4	Shapes of code	55
3.4.1	Spikes	55
3.4.2	Paragraphs	55
3.4.3	Paragraphs with headers	56
3.4.4	Suspicious comments	56
3.4.5	Intensive use of an object	56
3.5	Avoid instantiation for utility classes	57
3.6	Use immutable objects	58
3.7	Prefer records for immutability	62
3.8	Provide immutable decorators for sensitive mutable classes	64
3.9	Avoid constant interfaces	66

CONTENTS

3.10	Avoid global constant classes	67
3.11	Favour composition over inheritance	69
3.12	Use the <code>@Override</code> Annotation	72
3.13	Use the <code>@FunctionalInterface</code> Annotation	73
3.14	Prefer returning Null-Objects over <code>null</code>	74
3.15	Avoid <code>null</code> as method parameter	77
3.16	Prefer enhanced loops over for loops	79
3.17	Code against interfaces not implementations	80
3.18	Use existing exceptions	80
3.19	Validate method parameter	81
3.20	Prevent <code>NullPointerException</code> for String comparison	82
3.21	Safely cast <code>long</code> to <code>int</code>	83
3.22	Convert integers to floating point for floating-point math operations	84
3.23	Use Generics in favour of raw types	85
3.24	Prefer enums over <code>int</code> constants	86
3.25	Be aware of the contract between <code>equals</code> and <code>hashCode</code>	88
3.26	Use text blocks for multi-line strings	91
3.27	Use always braces for the body of all statements	93
3.28	Pre calculate the length in loops	94
3.29	Avoid slow instantiation of <code>String</code>	94
3.30	Use <code>StringBuilder</code> for concatenation	95
3.31	Reduce lookups in collection containers	96
3.32	Instantiate wrapper objects with <code>valueOf</code>	97
3.33	Use <code>entrySet</code> for iterating	98
3.34	Use <code>isEmpty()</code> for <code>String</code> length	99
3.35	Reduce the number of casts	99
4.	Software Quality Assurance	100
4.1	Test Pyramid	100
4.2	Test Classification	101
4.3	Test-driven Development (TDD)	102
4.4	Unit testing with JUnit 5	104
4.4.1	Unit Tests	104
4.4.2	JUnit 5	104
4.4.3	First unit test	106
4.4.4	Assertions	107
4.4.4.1	<code>assertEquals()</code> / <code>assertArrayEquals()</code>	107
4.4.4.2	<code>assertSame()</code> / <code>assertNotSame()</code>	107
4.4.4.3	<code>assertTrue()</code> / <code>assertFalse()</code> / <code>assertAll()</code>	108
4.4.4.4	<code>assertNull()</code> / <code>assertNotNull()</code>	108
4.4.4.5	<code>assertThrows()</code>	108
4.4.4.6	<code>assertTimeout()</code>	109
4.4.4.7	<code>fail()</code>	109
4.4.5	Annotations	110
4.4.5.1	<code>@Test</code>	111
4.4.5.2	<code>@BeforeEach</code> / <code>@AfterEach</code>	112

CONTENTS

	4.4.5.3	@BeforeAll / @AfterAll	113	
	4.4.5.4	@Disabled	114	
	4.4.5.5	@DisplayName	115	
	4.4.5.6	@Tag	115	
	4.4.5.7	@Timeout	116	
4.4.6	Assumptions		117	
	4.4.6.1	assumeFalse()	117	
	4.4.6.2	assumeTrue()	117	
	4.4.6.3	assumingThat()	117	
4.4.7	Parameterized Tests		118	
	4.4.7.1	@ValueSource	118	
	4.4.7.2	@MethodSource	119	
	4.4.7.3	@CsvSource	120	
	4.4.7.4	@CsvFileSource	120	
	4.4.7.5	@ArgumentsSource	121	
4.5	More on Unit Tests		122	
	4.5.1	Heuristics	122	
	4.5.2	Naming of test methods	125	
	4.5.3	Object Mother	126	
	4.5.4	Test Data Builder	127	
	4.5.5	F.I.R.S.T	129	
4.6	Mocking with Mockito		130	
	4.6.1	Types of Test Double	130	
	4.6.2	Activation	131	
	4.6.3	Annotations	132	
		4.6.3.1 @Mock	132	
		4.6.3.2 @Spy	133	
		4.6.3.3 @Captor	133	
4.7	Code Coverage		134	
4.8	Static Code Analysis		136	
4.9	Continuous Integration		137	
	4.9.1	Differences between CI, CD and CD	138	
	4.9.2	CI Workflow	140	
	4.9.3	Preconditions	141	
	4.9.4	Advantages and Disadvantages	142	
	4.9.5	Best Practices	142	
5.	Design Principles		143	
	5.1	Goal of Design Principles	143	
	5.2	Overview of Design Principles	143	
	5.3	SOLID Principles	145	
		5.3.1 Single Responsibility Principle	145	
			5.3.1.1 Example: Modem	145
			5.3.1.2 Example: Book	148
		5.3.2 Open Closed Principle	150	
			5.3.2.1 Example: LoanRequestHandler	150

CONTENTS

	5.3.2.2	Example: Shape	153
	5.3.2.3	Example: HumanResourceDepartment	155
	5.3.2.4	Example: Calculator	157
	5.3.2.5	Example: FileParser	159
5.3.3		Liskov Substitution Principle	161
	5.3.3.1	Example: Rectangle	161
	5.3.3.2	Example: Coupon	164
5.3.4		Interface Segregation Principle	168
	5.3.4.1	Example: MultiFunctionDevice	168
	5.3.4.2	Example: TechEmployee	171
5.3.5		Dependency Inversion Principle	174
	5.3.5.1	Example: UserService	174
	5.3.5.2	Example: Logger	175
5.4		Packaging Principles - Cohesion	177
	5.4.1	Release Reuse Equivalency Principle	177
	5.4.2	Common Closure Principle	177
	5.4.3	Common Reuse Principle	178
5.5		Packaging Principles - Coupling	178
	5.5.1	Acyclic Dependencies Principle	178
	5.5.1.1	Example: Cyclic dependency	179
	5.5.2	Stable Dependencies Principle	182
	5.5.3	Stable Abstractions Principles	184
5.6		Further Design Principles	187
	5.6.1	Speaking Code Principle	187
	5.6.2	Keep It Simple (and) Stupid!	187
	5.6.3	Don't Repeat Yourself / Once and Only Once	187
	5.6.4	You Ain't Gonna Need It!	188
	5.6.5	Separation Of Concerns	188
6.		Design Patterns of the Gang of Four	189
6.1		Creational	192
	6.1.1	Singleton	192
	6.1.1.1	Example: Lazy loading	192
	6.1.1.2	Example: Eager loading	193
	6.1.1.3	Example: Enum singleton	193
	6.1.2	Builder	194
	6.1.2.1	Example: MealBuilder	195
	6.1.2.2	Example: PizzaBuilder	197
	6.1.2.3	Example: Email	199
	6.1.2.4	Example: ImmutablePerson	201
	6.1.3	Factory Method	204
	6.1.3.1	Example: Logger	204
	6.1.3.2	Example: Department	206
	6.1.4	Abstract Factory	209
	6.1.4.1	Example: Car	210
	6.1.5	Prototype	215

CONTENTS

	6.1.5.1	Example: Person - Shallow copy	215
	6.1.5.2	Example: Person - Deep copy	218
	6.1.5.3	Example: Person - Copy constructor / factory	220
6.2	Structural		223
	6.2.1	Facade	223
	6.2.1.1	Example: Travel	224
	6.2.2	Decorator	225
	6.2.2.1	Example: Message	226
	6.2.2.2	Example: Window	227
	6.2.3	Adapter	230
	6.2.3.1	Example: Sorter	231
	6.2.3.2	Example: TextFormatter	232
	6.2.4	Composite	234
	6.2.4.1	Example: Graphic	235
	6.2.4.2	Example: Organization Chart	237
	6.2.5	Bridge	239
	6.2.5.1	Example: Message	239
	6.2.5.2	Example: Television	242
	6.2.6	Flyweight	245
	6.2.6.1	Example: Font	246
	6.2.6.2	Example: City	248
	6.2.7	Proxy	251
	6.2.7.1	Example: Spaceship	251
	6.2.7.2	Example: ImageViewer	253
6.3	Behavioural		255
	6.3.1	State	255
	6.3.1.1	Example: MP3Player	255
	6.3.1.2	Example: Door	258
	6.3.2	Template Method	261
	6.3.2.1	Example: Compiler	261
	6.3.2.2	Example: Callbackable	262
	6.3.3	Strategy	266
	6.3.3.1	Example: Compression	266
	6.3.3.2	Example: LogFormatter	268
	6.3.4	Observer	271
	6.3.4.1	Example: DataStore	272
	6.3.4.2	Example: Influencer	273
	6.3.5	Chain of Responsibility	275
	6.3.5.1	Example: Purchase	275
	6.3.5.2	Example: Authentication	278
	6.3.6	Command	281
	6.3.6.1	Example: FileSystem	281
	6.3.6.2	Example: Television	284
	6.3.7	Interpreter	287
	6.3.8	Iterator	288
	6.3.9	Mediator	289

Preface

This is a *Forever Edition*. That means that this book will see periodic updates.

If you purchased this book, thank you very much! Writing a book like this takes some hours of effort. Please treat your book as your own personal copy and do not redistribute the book without authorization.

If you find this book useful, I would greatly appreciate you purchasing a copy. By doing so, you'll be letting me know that books like this are useful to you.

What is the goal of this book?

Readers of this book will acquire in-depth knowledge and skills for the analysis, assessment and improvement of software quality. You will be able to apply principles, patterns, techniques and tools needed to write clean code.

Understanding important concepts of software quality in Java projects:

- Best Practices
- Functional and technological scope
- Development and programming models
- Tools

Learning Objectives:

- Acquisition of knowledge and skills to analyze, assess and improve software quality
- Learning principles, patterns, techniques and tools
- Deepening technology knowledge

Which topics can you expect?

Software testing is not limited to a specific phase of a project. Already during the coding phase or the build process, critical and difficult to find software defects can be detected in the source code. The necessary procedures and tools are presented in this book.

Among other things, the following topics will be covered:

- Overview of the basics of software quality
- Software metrics, metric application in practice
- Structured design, cohesion and coupling
- Overview of Principles, Best Practices and Code Smells

- Compliance and verification of Java code conventions
- Static software testing, especially review techniques and static program analysis
- Ensure software quality with tools such as SonarQube, PMD, SpotBugs, Checkstyle, ArchUnit and Dependency-Track
- Software tests with JUnit and Mockito
- Checking the test code coverage
- CI/CD
- Design Principles
- Design Patterns (GoF)

Who should read this book?

I wrote the book, because I want to help developers getting experience in clean code or improve it. Get them a feeling what it is about to be a Software Craftsman. It is about Professionalism, Pragmatism and Pride!

It's a mindset, where software developers choose to be responsible for their own careers, learning new tools and techniques and constantly getting better. The journey to a Software Craftsmanship is a long one.

That's why I wanted to share some techniques and practises for improving code quality and motivate developers.

What you must learn about Software Development yourself?

- **Developers are always wrong** - only the degree to which different developers are wrong in their endless architecture discussions differs!
- **If something can break, it will break** - make sure that a project is fool proof!
- **All code is bad** - there are only gradual gradations of the defectiveness of a piece of software!
- **There is always a bug somewhere** - you just must search long enough!
- **The most important thing is the customer** - and they don't give a damn about the technologies and processes used!
- **Project development on paper does not work** - problems are only identified during the development process!
- **Less is more** - if something is not necessary, just leave it out!
- **Only 20% of our work consists of coding** - the rest is designing, debugging, testing, meetings, discussing etc.
- **The customer never knows what he wants** - everyone has only a vague idea of the desired result!
- **Someone has done it before** - don't reinvent the wheel!

What about the code examples and typographic conventions?

All the code examples in this book are in Java. I don't want to put much boilerplate code within the code examples, so I just leave it out.

Class names, data types, method and function names and code are not included in proportional font.

Which literature is this book based on?

Schneider, Kurt: *Abenteuer Software Qualität – Grundlagen und Verfahren für Qualitätssicherung und Qualitätsmanagement*, dpunkt.verlag, 2007

Robert, Martin: *Clean Code – A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2009

Lilienthal, Carola: *Sustainable Software Architecture. Analyze and Reduce Technical Debt*, Dpunkt Verlag, 2019

Bloch, Joshua: *Effective Java – Third Edition*, Addison-Wesley, 2017

Roock, Stefan: *Refactorings in grossen Softwareprojekten*, Dpunkt Verlag, 2004

Gamma, Erich: *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

Robert C. Martin: *Agile Software Development: Principles, Patterns and Practices*, Prentice Hall, 2003

Bugayenko, Yegor: *Elegant Objects Volume 1*, CreateSpace Independent Publishing Platform, 2016

Bugayenko, Yegor: *Elegant Objects Volume 2*, CreateSpace Independent Publishing Platform, 2017

Harrer, Simon: *Java by Comparison: Become a Java Craftsman in 70 Examples*, O'Reilly UK Ltd., 2018

Robert, Martin: *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017

Robert, Martin: *The Clean Coder: A Code of Conduct for Professional Programmers*, Prentice Hall, 2011

David, Thomas: *Pragmatic Programmer special 2nd*, Addison-Wesley Professional, 2019

Kaczanowski, Tomek: *Practical Unit Testing with TestNg*, 2012

Meszaros, Gerard: *xUnit Test Patterns*, Addison-Wesley, 2007

Long, Fred: *Java Coding Guidelines: 75 Recommendations for Reliable and Secure Programs*, Addison-Wesley Professional, 2013

Sandro, Mancuso: *The Software Craftsman: Professionalism, Pragmatism, Pride*, Pearson, 2014

Freeman, Steve: *Growing Object-Oriented Software, Guided by Tests*, Addison-Wesley, 2009

1. Introduction to Software Craftmanship and Clean Code

1.1 A Passion for Software Development

Software Craftmanship is a long journey to mastery. It's a mindset where software developers choose to be responsible for their own careers, constantly learning new tools and techniques and constantly bettering themselves.

–Sandro Mancuso, The Software Craftsman

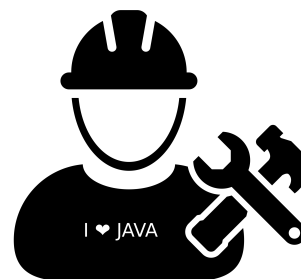
The basis of the Software Craftmanship movement is the publication of *Clean Code* by Robert C. Martin. The author is a co-founder of the Software Craftmanship movement. There are many initiatives like the *Manifesto for Software Craftmanship* or the *Clean Code Developer* movement which raised the bar regarding software development - luckily.

Software Craftmanship focuses on the adoption of good technical practices in the development process. It is obvious, that everyone likes Software Craftmanship. Everybody wants to have a clean code base, but it's difficult to achieve. There are several steps that should also be done in the right way. The practices defined under the Software Craftmanship movement are the ones that will help you to achieve this.

Software Craftmanship should be the goal you are aiming for. It is not only the maturity of your project, it is the quality of your software and your code base as well. As Benjamin Franklin said: *The bitterness of poor quality remains long after the sweetness of low price is forgotten.*

There are developers who are enthusiastic about their work and have already acquired these values to provide quality and kindness to the code. They have the necessary skills to realise products according to their values. Unfortunately, many others know little or not at all that there is another way to develop software. Even today, many development practices that help to create well-crafted software are simply not taught in schools.

If you work in an environment where these practices simply are not applied or there is no room for well-crafted software, ask you the question if you can change this. Be a craftsman, spread the professionalism of software development out to the team with the hope that individuals will follow you. Adopt the culture where the values of Software Craftmanship have the appropriate value. Do not rank other developers, maybe they had never the change to work in this way or had never the luck having a good mentor. When you have success with this approach, fine. If not, you may be in the wrong company for you!



Software Craftmanship

Practices that helps you strengthen the passion as a software developer:**Understand the importance of *good enough*.**

A working good enough solution is usually better than a non-existent perfect solution. Learn to balance constraints. Don't get bogged down in the war of beliefs about technologies and the right solution, be pragmatic and effective in what you do. Know where and when to compromise and when to say *enough*.

If you get stuck, don't hesitate to ask for help.

Don't spend your time endlessly trying to solve a problem that can easily be solved by someone else. There are people in your team that can you help to solve your problem much faster and give you valuable tips. Know when to keep trying and when to stop and ask for help.

If you are not able to communicate your ideas and connect with others, you will not get far.

Learn how to communicate effectively at every level with the people around you. Get involved with other ideas, listen to them and expand your horizons with other points of view. This will help you to question your own views and, in the end, to communicate them to others.

You will be criticised. Learn how to deal with it.

Understand the value of criticism and learn how to make the most of it. These are opportunities for you to grow and should be valuable feedback provided for you.

If you want to make a difference, you need to be effective in prioritising.

Not everything you can do is always worth doing. Understand how to identify high-impact work and dismiss the rest. Understand the basics and concepts and deepen them. Focus on a few things that you are very interested in while keeping track of other things. Nowadays frameworks come and go, it is more important to understand the concepts behind them. You can apply these to all frameworks coming.

Learn how you be effective in learning.

Find out the best way for you to learn new things over time. Explaining things to others and sharing your knowledge is a very effective way to deepen your own knowledge and learn new things.

Be selective with your valuable time.

You cannot do all by yourself. If you learn how to delegate effectively, you can multiply your time for topics which provide more value to you. Be realistic with your goals, you can't know everything.

Share all you learn with everyone else.

Not only you will help others around you to become better, but you will also be strengthening your own understanding and growth your knowledge. When knowledge grows in a team and exchange takes place, everyone wins.

Learn how to motivate others.

If you know how to light a fire and keep it alive, everyone will want to stay around you. Allow your team members to work in their own way and give them the freedom they need to work

creatively and effectively. Set a good example and find out what your colleagues need to be motivated.

Learn to be proactive.

Don't wait for problems to become problems. Take care of things in a proactive way.

1.2 Manifesto for Software Craftmanship

The famous [Manifesto for Software Craftmanship](http://manifesto.softwarecraftmanship.org/)¹ describe professional software developers, and itself reads like an addition to the [Agile Manifesto](http://agilemanifesto.org/)²:

As aspiring Software Craftsmen, we are raising the bar of professional software development by practicing it and helping others learn the craft. Through this work we have come to value:

- Not only working software, but also **well-crafted software**
- Not only responding to change, but also **steadily adding value**
- Not only individuals and interactions, but also **a community of professionals**
- Not only customer collaboration, but also **productive partnerships**

That is, in pursuit of the items on the left we have found the items on the right to be indispensable.

1.3 Clean Code Developer

The [Clean Code Developer \(CCD\)](https://clean-code-developer.com/)³ is an initiative for more professionalism in software development. CCD define a collection of principles and practices. These are divided into different degrees. However, it also includes a collection of specific values, such as **evolvability**, **correctness**, **production efficiency** and **continuous improvement**.

Through constant iteration of the different degrees, these principles and practices are becoming part of the way we act. This creates an awareness of quality within the development process. Those who practice daily will be able to intuitively retrieve this knowledge

Below you can see an overview of the degrees of the CCD system, which the iterative cycle goes through. The degrees build on each other in terms of difficulty and benefits. A developer who has not yet started with CCD owns the black grade. If someone not yet started with CCD own the black grade. A black grade only indicates the initial interest in CCD.

¹<http://manifesto.softwarecraftmanship.org/>

²<http://agilemanifesto.org/>

³<https://clean-code-developer.com/>